

Project Report: Week 5

Implementation Summary & Code Revision

1. Objective & Architecture Overview

The core focus of Week 5 was implementing an end-to-end data pipeline, feature engineering framework, and deep learning network using an absolute implementation file. The script allows programmatic ingestion of customizable financial assets using dynamic inputs, computes five core indicators, generates temporal sliding vectors, trains an LSTM network, and calculates a recursive 15-step directional future forecast.

2. Implementation & Code Revision Breakdown

Task 1: Data Collection Pipeline

This module establishes connection properties with external endpoints to dynamically parse ticker strings, custom frame horizons, and sample intervals.

Python

```
import yfinance as yf

# Fetch data dynamically via user queries
stock_df = yf.download(ticker, start=start_date, end=end_date, interval=interval)

if isinstance(stock_df.columns, pd.MultiIndex):
    stock_df.columns = stock_df.columns.get_level_values(0)
```

Code Term Breakdown:

- `yf.download(...)` : Connects directly to the Yahoo Finance API to return a structured multi-column historical dataset tailored to target constraints (e.g., Daily, Weekly, or Monthly frequencies).
- `stock_df.columns.get_level_values(0)` : Drops multi-ticker structural index layers returned by `yfinance` to standardize flat dataframe column lookups (`'Open'` ,

'High', 'Low', 'Close', etc.).

Task 2: Multi-Variable Indicator Overlay & Visualization

Extracts raw transaction closing values and constructs multiple technical layers reflecting simple momentum, exponential trend followings, divergence spreads, and bounded momentum oscillators.

Python

```
# 1. Simple & Exponential Smoothers
stock_data['SMA_5'] = stock_data['Close'].rolling(window=5).mean()
stock_data['EMA_5'] = stock_data['Close'].ewm(span=5, adjust=False).mean()

# 2. Moving Average Convergence Divergence (MACD)
ema12 = stock_data['Close'].ewm(span=12, adjust=False).mean()
ema26 = stock_data['Close'].ewm(span=26, adjust=False).mean()
stock_data['MACD'] = ema12 - ema26
stock_data['Signal'] = stock_data['MACD'].ewm(span=9, adjust=False).mean()

# 3. Relative Strength Index (RSI)
delta = stock_data['Close'].diff()
gain = delta.where(delta > 0, 0)
loss = (-delta).where(delta < 0, 0)
avg_gain = gain.rolling(window=14, min_periods=14).mean()
avg_loss = loss.rolling(window=14, min_periods=14).mean()
rs = avg_gain / avg_loss.replace(0, np.nan)
stock_data['RSI_14'] = (100 - (100 / (1 + rs))).fillna(50)
```

Code Term Breakdown:

- `.rolling(window=5).mean()` : Generates a rolling window computation computing the arithmetic mean across the most recent 5 steps.
- `ema12 - ema26` : Computes the **MACD Line**, capturing directional shifts by subtracting a slow 26-period EMA from a fast 12-period EMA.
- `stock_data['MACD'].ewm(span=9...)` : Computes the MACD **Signal Line** using a 9-period EMA loop to generate trigger crossover markers.
- `delta.where(delta > 0, 0)` : Isolates localized upward price fluctuations while setting negative down-periods to 0, forming the mathematical foundation for calculating the Relative Strength Index over a 14-period trading horizon.

Task 3: Data Preprocessing & Sliding Window Vectors

Converts historical data scales into standardized tensors and maps raw vectors into lookback sequences.

Python

```
from sklearn.preprocessing import MinMaxScaler

feature_cols = ['Close', 'SMA_5', 'EMA_5', 'MACD', 'RSI_14']
raw_features = stock_data[feature_cols].values

feature_scaler = MinMaxScaler(feature_range=(0, 1))
scaled_features = feature_scaler.fit_transform(raw_features)

X, y = [], []
for i in range(lookback_window, len(scaled_features)):
    X.append(scaled_features[i-lookback_window:i])
    y.append(scaled_features[i, 0])
X, y = np.array(X), np.array(y)
```

Code Term Breakdown:

- `MinMaxScaler(feature_range=(0, 1))` : Standardizes variations by bounding value columns within a consistent 0 to 1 range, protecting optimization parameters from scale anomalies.
- `X.append(scaled_features[i-lookback_window:i])` : Extracts a 60-row historical window sequence slice spanning all 5 indicator fields to serve as the input tensor, matching target vectors tracking the single subsequent adjusted close price.

Task 4: Building and Training the LSTM Model

Assembles a computational neural structure built specifically for continuous sequence tracing.

Python

```
from keras.models import Sequential
from keras.layers import LSTM, Dense, Dropout

model = Sequential()
model.add(LSTM(units=50, return_sequences=False, input_shape=(timesteps, features)))
model.add(Dropout(0.2))
model.add(Dense(units=1))
model.compile(optimizer='adam', loss='mean_squared_error')
```

Code Term Breakdown:

- `LSTM(units=50, return_sequences=False)` : Instantiates a recurrent layer with 50 nodes. `return_sequences=False` instructs the layer to return only the final internal hidden representation vector, flattening structural tensors before passing them downstream.
- `Dropout(0.2)` : Randomly deactivates 20% of network nodes during optimization steps to reduce parameter interdependence and prevent overfitting.
- `'mean_squared_error'` : Evaluates the model's loss by calculating the average squared difference between predictions and actual labels.

Task 5: Model Evaluation & Multi-Step Future Forecasting

Processes training metrics using structural accuracy scoring and projects market values beyond the available dataset using a recursive forecasting loop.

Python

```
from sklearn.metrics import r2_score

# 1. R² Evaluation
r2 = r2_score(actual_prices, train_predictions)

# 2. Autoregressive Rolling Future Forecast
future_forecast_scaled = []
for _ in range(forecast_steps):
    next_pred_scaled = model.predict(current_batch)[0, 0]
    future_forecast_scaled.append(next_pred_scaled)

    last_indicators = current_batch[0, -1, 1:]
    next_feature_row = np.hstack([next_pred_scaled, last_indicators]).reshape(1, 1,
X.shape[2])
    current_batch = np.append(current_batch[:, 1:, :], next_feature_row, axis=1)
```

Code Term Breakdown:

- `r2_score(...)` : Evaluates the model's performance by calculating the Coefficient of Determination, measuring how well the model captures target variance.
- `np.append(current_batch[:, 1:, :], next_feature_row, axis=1)` : Drops the oldest sequence slice from the active data batch and appends the newly predicted

closing estimate along with static trailing indicator parameters (`last_indicators`). This updates the 60-day sliding window array to recursively project future time intervals.

- `plt.savefig(...)` : Saves the finalized visualization plot (containing historical actuals, model fits, and the predicted 15-unit future trajectory) using a unique ticker and timestamp filename string.